



INSTALLATION PROJECT 1

CODY LANCASTER
ARTI 405



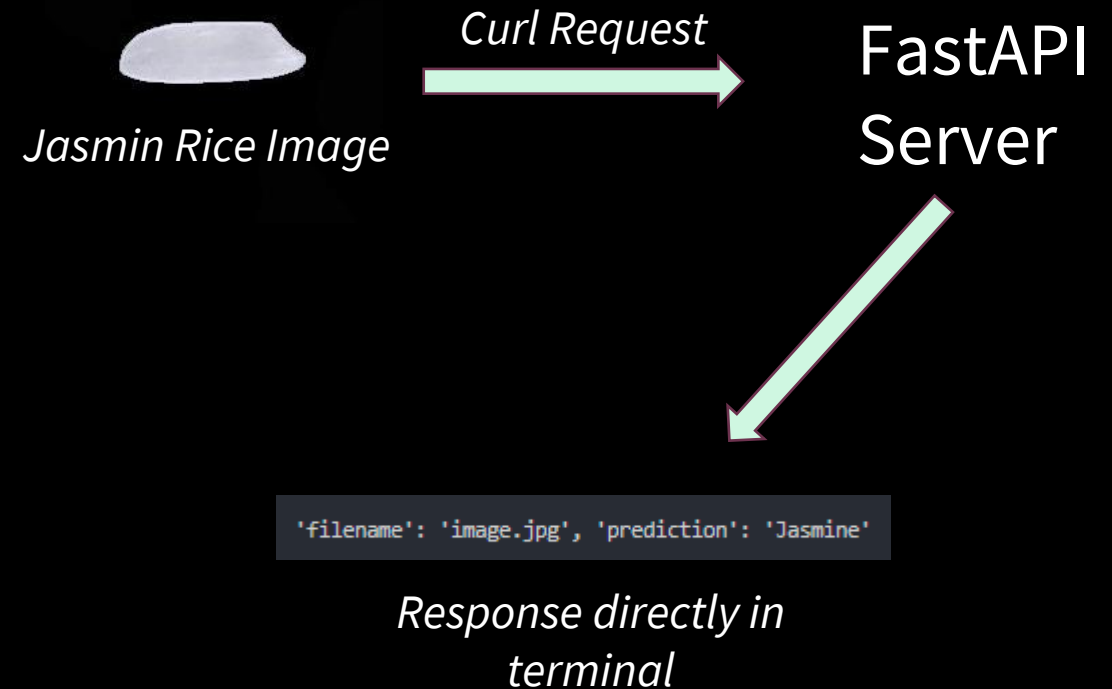
● Project Overview

Goal: - Deploy a usable, custom machine learning model

Method: - I am going to use Docker

Model: - I am going to use a model made in conjunction with Logan for ARTI 450, which classifies types of rice grains, from images.

Example Usage

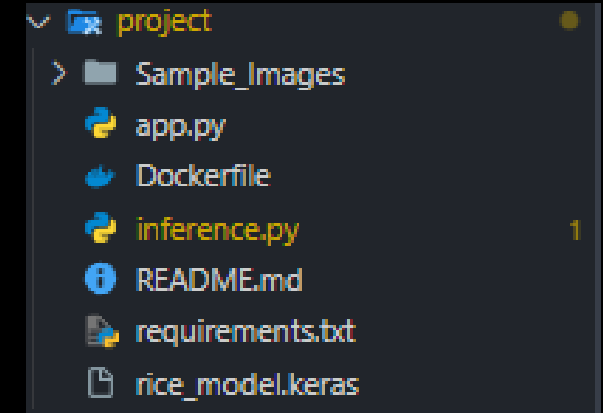


● General Outline

The general process is going to look something like this:

1. Make a Project environment folder
2. Install Docker Desktop
3. Build and Train the model
4. Write server (and client-side code)
5. Write Dockerfile -> Build Docker Image -> Run Docker Image
6. Communicate at will, via Curl requests

File Structure:



For the purpose of demonstration, this API server and Docker image run off of a VM on my local machine.

Dependencies:

```
1 tensorflow==2.18.0
2 keras==3.8.0
3 scikit-learn==1.6.1
4 fastapi
5 uvicorn[standard]
6 pillow
7 python-multipart
```

Very Lightweight!



Docker Setup

Docker File:

```
# Use a minimal Python image
FROM python:3.11-slim

WORKDIR /app

# Copy only necessary files
COPY rice_model.keras ./
COPY inference.py ./
COPY app.py ./
COPY requirements.txt ./

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Expose port for FastAPI
EXPOSE 8000

# Run the FastAPI app with Uvicorn
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Build & Run:

```
> docker build -t rice-classifier .
```

```
> docker run -d --name rice-classifier -p 8000:8000 rice-classifier
```

Docker Image:

☐	☒	rice-classifier	latest	950ca6a124d1	43 minutes ag	3.32 GB
--------------------------	-------------------------------------	-----------------	--------	--------------	---------------	---------

Docker Container:

rice-classifier

12fa1f748ddc rice-classifier:latest 8000:8000

STATUS
Running (4 minutes ago)

Logs

Inspect

Bind mounts

Exec

Files

Stats

2025-04-19 22:20:48

2025-04-20 02:20:48.145142: I external/local_xla/xla/tsl/cuda/cudart_stub.cc:32] Could not find cuda drivers on your machine, GPU will not be used.

2025-04-19 22:20:48

2025-04-20 02:20:48.147877: I external/local_xla/xla/tsl/cuda/cudart_stub.cc:32] Could not find cuda drivers on your machine, GPU will not be used.

2025-04-19 22:20:48

2025-04-20 02:20:48.153133: E external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:477] Unable to register cuFFT factory: Attempting to register factory for plugin cuFFT when one has already been registered

2025-04-19 22:20:48

WARNING: All log messages before absl::InitializeLog() is called are written to STDERR

2025-04-19 22:20:48

E0000 00:00:1745115648.163891 1 cuda_dnn.cc:8310] Unable to register cuDNN factory: Attempting to register factory for plugin cuDNN when one has already been registered

2025-04-19 22:20:48

E0000 00:00:1745115648.167024 1 cuda_blas.cc:1418] Unable to register cuBLAS factory: Attempting to register factory for plugin cuBLAS when one has already been registered

2025-04-19 22:20:48

2025-04-20 02:20:48.179196: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.

2025-04-19 22:20:48

To enable the following instructions: AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.

2025-04-19 22:20:50

2025-04-20 02:20:50.493980: E external/local_xla/xla/stream_executor/cuda/cuda_driver.cc:152] failed call to cuInit: INTERNAL: CUDA error: Failed call to cuInit: UNKNOWN ERROR (303)

2025-04-19 22:20:50

/usr/local/lib/python3.11/site-packages/keras/src/saving/saving_lib.py:757: UserWarning: Skipping variable loading for optimizer 'rmsprop', because it has 12 variables whereas the saved optimizer has 22 variables.

2025-04-19 22:20:50

saveable.load_own_variables(weights_store.get('inner_path'))

2025-04-19 22:20:50

INFO: Started server process [1]

2025-04-19 22:20:50

INFO: Waiting for application startup.

2025-04-19 22:20:50

INFO: Application startup complete.

2025-04-19 22:20:50

INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)

Code

The code that's used for the API is extremely lightweight, using only **53** lines of code between the actual app and the helper module.

App File

```
from fastapi import FastAPI, File, UploadFile, HTTPException
from inference import predict
import os

app = FastAPI()

tmp_dir = '/tmp'
if not os.path.isdir(tmp_dir):
    os.makedirs(tmp_dir)

@app.post("/predict")
async def predict_image(file: UploadFile = File(...)):
    # Save upload to temporary file
    try:
        file_path = os.path.join(tmp_dir, file.filename)
        contents = await file.read()
        with open(file_path, 'wb') as f:
            f.write(contents)
        # Run prediction
        result = predict(file_path)
        return {"filename": file.filename, "prediction": result}
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e))
```

Helper Module

```
import numpy as np
from PIL import Image
import tensorflow as tf

# Load the trained Keras model (ensure rice_model.keras is in the same directory)
model = tf.keras.models.load_model('rice_model.keras')

# Mapping from model output index to rice class name
class_names = ['Ipsala', 'Jasmine', 'Arborio', 'Karacadag', 'Basmati']

def preprocess_image(image_path, target_size=(50, 50)):
    """
    Load an image file, convert to grayscale, resize, normalize, and expand dims
    """
    img = Image.open(image_path).convert('L') # grayscale
    img = img.resize(target_size)
    arr = np.array(img) / 255.0
    arr = arr.reshape((target_size[0], target_size[1], 1))
    return np.expand_dims(arr, axis=0)

def predict(image_path):
    """
    Predict the class of a rice grain image.
    Returns the class name as a string.
    """
    input_arr = preprocess_image(image_path)
    preds = model.predict(input_arr)
    idx = int(np.argmax(preds, axis=1)[0])
    return class_names[idx]
```



Model

As mentioned in a previous slide, the model I'm using was built and trained by Logan and I. It boasts very respectable results.

Training Epochs (Avg: 0.97568)

Epoch 1/10	1875/1875	12s 4ms/step	- accuracy: 0.9042	- loss: 0.2551
Epoch 2/10	1875/1875	8s 4ms/step	- accuracy: 0.9687	- loss: 0.0926
Epoch 3/10	1875/1875	7s 4ms/step	- accuracy: 0.9732	- loss: 0.0773
Epoch 4/10	1875/1875	7s 4ms/step	- accuracy: 0.9793	- loss: 0.0602
Epoch 5/10	1875/1875	7s 4ms/step	- accuracy: 0.9838	- loss: 0.0474
Epoch 6/10	1875/1875	7s 4ms/step	- accuracy: 0.9866	- loss: 0.0389
Epoch 7/10	1875/1875	7s 4ms/step	- accuracy: 0.9880	- loss: 0.0329
Epoch 8/10	1875/1875	7s 4ms/step	- accuracy: 0.9896	- loss: 0.0292
Epoch 9/10	1875/1875	10s 4ms/step	- accuracy: 0.9908	- loss: 0.0273
Epoch 10/10	1875/1875	7s 4ms/step	- accuracy: 0.9926	- loss: 0.0205

Test Evaluation

469/469 - 3s - 6ms/step - accuracy: 0.9872 - loss: 0.0386

0.9872 - 0.97568 =

+0.01152



● Making a Request

The process of making a classification request to the API is extremely simple:

Here's the *Sample_Images* I'll be using:



Example Output:

```
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
100  3531  100    50  100   3481    1171   81562  --:--:--  84071{"filename":"Arborio1.jpg","prediction":"Arborio"}
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
100  3537  100    50  100   3487    1196   83423  --:--:--  86268{"filename":"Arborio2.jpg","prediction":"Arborio"}
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
100  3803  100    50  100   3753    1170   87828  --:--:--  90547{"filename":"Basmati1.jpg","prediction":"Basmati"}
```

I used a Git Bash to send the requests:

```
codyd@DESKTOP-2BDCCI7 MINGW64 /d/School/ARTI-405/installation project 1/project
$ for img in Sample_Images/*;
> do curl -F "file=@${img}" http://localhost:8000/predict;
> done
```

Since this runs off a local environment, I am positioned within the project folder.

- Prediction 1: Arborio
- Prediction 2: Arborio
- Prediction 3: Basmati





Interface Usage

Alternatively, the CLI interface can be used asynchronously in the terminal:

First run to initialize:

```
PS D:\School\ARTI-405\installation project 1\project> python cli.py
Interactive Rice Classifier. Type 'exit' to quit.
Enter path to rice grain image (or 'exit' to quit): Sample_Images/Arborio1.jpg
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1745537948.427950 20824 service.cc:152] XLA service 0x1cefb8aaf30 initialized for platform Host (this does not guarantee that XLA will be used). Devices:
I0000 00:00:1745537948.428208 20824 service.cc:160] StreamExecutor device (0): Host, Default Version
I0000 00:00:1745537948.553719 20824 device_compiler.h:188] Compiled cluster using XLA! This line is logged at most once for the lifetime of the process.
1/1 ██████████ 0s 291ms/step

Prediction: Arborio
```

Unlike making a curl request for bulk predictions, the CLI interface serves the purpose of doing single predictions in real-time.

Subsequent runs:

```
Enter path to rice grain image (or 'exit' to quit): Sample_Images/Arborio1.jpg
1/1 ██████████ 0s 20ms/step

Prediction: Arborio
Enter path to rice grain image (or 'exit' to quit): Sample_Images/Arborio2.jpg
1/1 ██████████ 0s 20ms/step

Prediction: Arborio
```

